Chelin Jebastin Raj

ERPNext / Frappe Developer

Bengaluru, India | cjraj2002@gmail.com | +91-8072988052 | linkedin.com/in/chelin-jebastin-raj

PROFESSIONAL SUMMARY

ERPNext / Frappe Developer with 2+ years of experience building and extending ERPNext for electronics refurbishment and manufacturing operations. Strong in Python and JavaScript on the Frappe framework: custom Doctypes, workflows, server and client scripts, background scheduler jobs, and large-scale automation engines for production planning, inventory sync, and order allocation. Experienced integrating ERPNext with external systems — Shopify (GraphQL / REST), a Django-based laptop testing and quality-inspection system (NJPulse), and warehouse barcode scanners — and comfortable running the production stack end to end: AWS EC2, Nginx, MariaDB performance tuning, and Linux administration. Works directly with business, warehouse, and manufacturing stakeholders to turn operational pain points into reliable, maintainable ERP solutions.

ERPNEXT / FRAPPE CORE COMPETENCIES

- Custom ERPNext modules, Doctypes, workflows, and field-level permissions tailored to manufacturing and business processes
- Server Scripts, Client Scripts, and custom apps in Python and JavaScript for validation, conditional UI, and task automation
- Frappe background scheduler jobs and queue-based batch processing — paginated, self-enqueueing, with Redis-backed distributed locking
- Production planning, serial-level order allocation, BOM / part allocation, Material Requests, and Stock Movement controls
- E-commerce and system integrations: Shopify GraphQL / API inventory and metafield sync, Django (NJPulse) ↔ ERPNext REST integration with caching
- MariaDB performance tuning — slow-query profiling, index optimization, InnoDB lock-contention resolution
- Production deployment and support: AWS EC2, Nginx reverse proxy, Linux server administration, log-based root-cause analysis

TECHNICAL SKILLS

ERP & Frameworks: Frappe Framework, ERPNext (Doctypes, Workflows, Permissions, Server / Client Scripts, Custom Apps, Scheduler Jobs, Manufacturing & Stock), Django 5, Django REST Framework, Flask

Languages: Python 3, JavaScript, SQL, Bash / Shell scripting, HTML5, CSS3

Databases & Caching: MariaDB, MySQL, PostgreSQL — query writing, performance tuning, index optimization, lock-contention resolution; Redis; Django DB cache

Integrations & APIs: Shopify GraphQL API, REST APIs, ERPNext / Frappe REST API, JSON, Paramiko (SFTP), Postman

Hardware & Media: OpenCV, PyDub, SpeechRecognition, FFmpeg / FFprobe, NumPy, Psutil

Cloud & Infra: AWS (EC2, S3, IAM, Security Groups, CloudWatch), Nginx, Gunicorn, Supervisor, Logrotate, Linux (Ubuntu) Server Administration

Dev Tools: Git, GitHub, Postman, VS Code, Linux

PROFESSIONAL EXPERIENCE

Software Engineer *Apr 2024 – Present*

NewJaisa Technologies Ltd. — Bengaluru, India

- Customized ERPNext modules — Doctypes, workflows, and field-level permissions — to adapt the out-of-the-box ERP system to the company's specific manufacturing and business processes as requirements evolved
- Developed Python and JavaScript scripts for real-time field validation, conditional UI control, and task automation, reducing manual data-entry errors and enforcing business rules directly inside ERPNext forms
- Developed NJPulse, a Django-based laptop diagnostics and quality-inspection system, and integrated it with ERPNext over REST so serial numbers, inspection grades, job cards, and stock movements stay synchronized between both systems
- Built automation engines on Frappe for production planning, order-to-serial allocation, and Shopify inventory sync, running as scheduled and queued background jobs against live warehouse and factory data
- Built scheduler-driven background jobs and batch processes inside Frappe for large-volume automation, using paginated processing, self-enqueueing jobs, and Redis-backed locking
- Managed AWS EC2 instances end-to-end for the production ERP environment, including provisioning, configuring Nginx as a reverse proxy in front of the application server, and maintaining MariaDB as the backing database
- Resolved recurring MariaDB performance issues — including InnoDB lock contention, long-running queries, and missing indexes — by profiling slow queries and applying targeted SQL tuning, restoring stable performance for production ERP workloads
- Administered Linux servers supporting the ERP stack — monitoring system logs, tracking resource usage, and maintaining uptime for a production environment relied on daily by business and manufacturing teams
- Collaborated directly with business and manufacturing stakeholders to translate operational pain points into ERP configuration and development changes, improving data accuracy and reducing manual workarounds across teams
- Took ownership of production issues end-to-end — from log-based diagnosis through root-cause identification to deployment of a fix — across both the ERPNext application layer and the underlying Linux / AWS infrastructure

KEY PROJECTS

Unified Production Planning & Intelligent Part Allocation Engine

- Built an automated production scheduling engine in Python / Frappe that orchestrates multi-stage laptop remanufacturing across 10+ factory departments, driven by cumulative transit SLAs and sales velocity
- Automated item-level production requirement mapping from Sales Orders, with custom Python + JavaScript scripts that trigger planning workflows on Sales Order submission, removing manual planning overhead
- Developed a 4-tier order-to-serial matching algorithm with physical compatibility gates (CPU generation, motherboard architecture, touch vs. non-touch screen) to align daily builds with actual order demand
- Engineered an automated Bill of Materials (BOM) allocation engine that optimizes dual-slot RAM and SSD permutations against live warehouse inventory, automatically raising nightly Material Requests (MRs) for assembly technicians
- Designed a "Reconfig Rescue" system that programmatically identifies units dwelling more than 30–60 days in WIP and reconfigures them into high-demand market SKUs, minimizing dead stock and capital lockup
- Integrated strict plan verification into warehouse barcode scanners (Stock Movement), blocking unauthorized or out-of-sequence physical transfers on the shop floor

Tech Stack: *Python, JavaScript, Frappe, ERPNext Manufacturing & Stock, MariaDB, Background Scheduler Jobs*

Automatic Order Allocation & Fulfillment Engine

- Architected an order-to-inventory allocation engine that automatically assigns physical serialized laptops and desktops to open customer orders across multiple warehouse locations
- Optimized fulfillment queue sequencing to prioritize orders by customer SLA and age (FIFO), preventing older orders from getting stranded and stopping unapproved B2B holds from locking retail stock
- Improved same-day delivery compliance by routing rush orders to shelf-ready packed stock rather than units still in Quality Check, and re-linked serviced / repaired units back to their original customers
- Built an automated daily re-allocation daemon for at-risk orders, plus a one-click manual re-allocation interface for customer support and fulfillment teams

Tech Stack: *Python, ERPNext, MariaDB, Background Queues*

Shopify ↔ ERPNext Sync Engine — Inventory, Delivery Dates & Product Metafields

- Engineered a bi-directional inventory synchronization engine between Shopify and ERPNext so live stock counts and delivery promise dates match physical warehouse availability
- Eliminated e-commerce overselling by resolving API rate-limit drops, preventing hourly syncs from relisting sold-out items, and separating pre-order pools from ready-to-ship stock
- Audited and cleaned the multi-channel product catalog, fixing 1,436 website listings mapped to deleted ERP items and correcting 353 listings showing inflated stock balances
- Established unified catalog rules for pre-order and "continue selling" states, standardizing dynamic delivery-date calculations across Shopify storefronts, ERP Sales Orders, and Sales Quotations
- Built the product metafield sync: a Python engine reading product data from ERPNext and pushing structured metafields to Shopify via the GraphQL API
- Designed a Frappe background scheduler job with a Redis-backed distributed lock — processes thousands of Product Bundles in paginated batches with automatic self-enqueue
- Handled Shopify GraphQL rate limiting with exponential back-off retry (up to 8 attempts), chunked metafield mutations (25 per call), and INVALID_TYPE error recovery
- Developed persona-matching logic comparing Item Variant Attributes against a Performance Config Criteria table to auto-assign product personas

Tech Stack: *Python, Frappe / ERPNext, Shopify API, Shopify GraphQL API, Redis, Background Scheduler Jobs, REST, JSON*

NJPulse — Laptop Diagnostics & Quality Inspection System (Django + ERPNext)

- Developed a laptop diagnostics and quality-inspection platform (Django 5, DRF) that digitizes laptop refurbishment, bench testing, multi-stage quality inspection, and repair workflows, integrated with ERPNext
- Built an automated hardware diagnostic suite: model-specific virtual keyboard tester (Standard, TKL, Chromebook, Chiclet, Gaming, Mechanical layouts loaded from JSON configs), display / dead-pixel and touchscreen checks, touchpad tests, OpenCV webcam verification, and microphone testing with PyDub and SpeechRecognition
- Implemented standardized Grade A–D inspection logic, review / override flow before submission to ERP, and a TimerService that tracks planned vs. actual inspection time to measure technician throughput
- Built Job Card and Machine Part Change (MPC) workflows that route Grade C / D units to repair queues and track serial-numbered component swaps (RAM, SSD, battery, display) against the parent laptop, plus a knowledge base of repair SOPs linked to laptop models
- Designed an asynchronous media pipeline that records inspection sessions and syncs large video files to a dedicated media server over SFTP (Paramiko), keeping bench storage clean and web performance stable
- Added mobile-routing middleware for handheld barcode scanners and tablets, with QR deep-links to the exact inspection or job-card screen
- Integrated with ERPNext REST APIs (Item, Serial No, Quality Inspection, Stock Movement, Delivery Note, Job Card) using token auth, ERP error unwrapping, and a Django DB cache with SHA256-hashed keys to cut redundant ERP calls during peak shifts

Tech Stack: *Python, Django 5, DRF, PostgreSQL, OpenCV, PyDub, SpeechRecognition, Paramiko, Gunicorn, Nginx, ERPNext API*

ERPNext Custom Modules — Audit, Registry & BOM Tracking *2024*

- Audit Doctype: built stock validation mechanism with automated cross-checks for inventory accuracy and full warehouse traceability
- Security Inward Registry: developed goods-in tracking module cross-referencing Purchase Orders and GRNs for vendor compliance
- Machine Part Changes: designed flexible BOM component tracking system to log and audit part substitutions across production runs

Tech Stack: *Python, JavaScript, ERPNext, Frappe, MariaDB*

EDUCATION

B.Tech – Electrical and Electronics Engineering *2019 – 2023*

UVOC College of Engineering, Thoothukudi

CERTIFICATIONS

- Python Full Stack Development — Pyspiders Institute (Certified)